Loop App · Loop Agents Consulting — Análise de sistema, PRD e FRD

Campo	Valor
Versão	0.1 (rascunho para revisão)
Data	2026-11-10
Repositório	`adm` (webapp de planejamento; conecta-se ao repositório de cada cliente)
Primeiro cliente	CleanSea (coleta de resíduos de navios)
Método	`engineering:system-design` (requisitos → alto nível → detalhamento → escala → trade-offs)
Fontes analisadas, na ordem pedida	1. *Loop Agents Consulting* (notas) · 2. *Baptista — 27 blocos* · 3. PDF de slides técnicos (University of Hull, 25 págs.) · 4. *DIAGRAMA_V2.html* (Mapa de Fluxos CleanSea)

Legenda de evidência (a mesma do Mapa de Fluxos): **[A]** declarado pela empresa · **[B]** referência externa · **[C]** hipótese a validar · **[D]** dado desconhecido. Valores marcados **A DEFINIR** não foram inventados: precisam vir do cliente ou de uma decisão.

---

0. Resumo em uma página

Pergunta	Resposta
O que é?	Uma plataforma da Loop Agents Consulting para estruturar a empresa-cliente como um sistema, simular cenários com ela e gerenciar a implementação de agentes de IA, ligada ao repositório GitHub do projeto do cliente. [A]
Para quem?	Desenvolvedor/consultor da Loop (área privada) e cliente (área compartilhada, studio e proposta). Primeiro cliente: CleanSea. [A]
Por que existe?	O primeiro valor que converte uma proposta em serviço é deixar o cliente **ver a própria empresa e interagir com cenários e simulações**. [A]
Por que agora?	CleanSea já tem um protótipo de mapa de fluxos e um plano de 10 entregas (E_1 a E_10). O repositório `adm` já tem monorepo, design de diagrama CLD e pipeline de dados. [A]
O que muda?	A proposta deixa de ser um PDF estático e passa a ser um modelo vivo: formulário → diagrama → simulação → decisão → backlog no repositório do cliente. [C]
O plano	Fase 0 fundação (auth, tenants, template de projeto) → Fase 1 Loop Diagrama (formulário + mapa + simulação) → Fase 2 Studio e proposta → Fase 3 operação de agentes CleanSea (A0–A3, WhatsApp, MCP, analytics). §4.6
O que precisamos do cliente	Lista do João (§4.8): fluxo atual do pedido, volumes, tempos, regras por porto, aprovadores, canais (e-mail, WhatsApp Business), KPIs e incidentes caros. [D]
Como saberemos que deu certo	Cliente aprova a proposta pelo modelo; ciclo pedido→cotação medido antes/depois; parâmetros do mapa trocados de fictícios para [A]/[B]. §4.5
Investimento	**A DEFINIR** (depende das respostas do cliente e da escolha de infraestrutura, §3.8).

---

1. Análise das fontes

1.1 Loop Agents Consulting (notas do fundador)

Fatos declarados [A]

- Loop Consulting desenvolve, implementa e gerencia agentes de IA para empresas.
- A plataforma conecta-se diretamente ao repositório do projeto do cliente.
- Uso múltiplo: comunicação, desenvolvimento, visualização, engenharia, arquitetura.
- Quatro áreas:
    1. Gerenciamento estruturado do projeto — **somente desenvolvedor**.
    2. Área compartilhada desenvolvedor ↔ cliente.
    3. **Studio / showroom** da solução.
    4. **Proposta comercial** e visualização do projeto.
- Método: "Loop Method" e "Loop App"; pensamento sistêmico, dinâmica de sistemas, **diagramas de influência**, **arquétipos**, Stafford Beer / **Viable System Model (VSM)**, métodos de estruturação de problemas (PSMs), Critical Systems Thinking, julgamentos de fronteira, relação entre participantes (unitária, pluralista, coerciva).
- Workflow inicial: **abrir o projeto na plataforma → espelhar e sincronizar com o repositório do cliente**.
- O projeto do cliente **nasce de template e schemas padronizados** e é personalizado de forma interativa.
- O cliente recebe **login** para fornecer informações e acompanhar o desenvolvimento.
- Plataforma por cliente: **rota específica + senha**; o cliente usa durante todo o ciclo (andamento, dúvidas).
- "Loop Diagrama" é a fase inicial: o cliente responde ao formulário, vê os diagramas sendo construídos e roda simulações.
- Tarefas transversais: meta-processos contínuos, check-in/out da produção, templates de schemas, eficiência de IA/modelos, RAG/busca semântica, banco de dados, hospedagem, e-mail, WhatsApp (Meta).

CleanSea — plano declarado [A]

ID	Entrega
E_1	Webapp próprio no domínio da CleanSea + app Expo para download
E_2	4 agentes: A0 Orquestrador, A1 Triagem, A2 Extração e rascunho, A3 FAQ
E_3	MCP próprio
E_4	Integração de dados dinâmicos via API (3 sites)
E_5	Dashboard analítico com preditiva, tendências e insights
E_6	Integração com WhatsApp
E_7	Sistema de UI de formulários
E_8	Acesso ao Loop App integrado ao app CleanSea
E_9	Plugin
E_10	Onboarding Prisma

Nota: "iniciar com protótipo de **um porto**" [A]; a lista de perguntas cita **28 portos** [A].

Lacunas e riscos identificados

- As quatro áreas não têm regras de acesso detalhadas (quem vê o quê, quando uma informação passa da área privada para a compartilhada). → FRD §5.2.
- "Rota específica + senha" sugere senha compartilhada por cliente. Isso é fraco para dados de operação; recomendação: login individual por pessoa (link mágico/OAuth) dentro do tenant do cliente. → §3.6.
- "Espelhar e sincronizar com o repositório" não define direção, frequência nem conflito. → §3.4 e FRD §5.3.
- E_1 a E_10 misturam produto da Loop (Loop App) e produto da CleanSea (app, agentes). O PRD separa os dois escopos (§4.3).

1.2 Baptista — 27 blocos (pensamento sistêmico na prática)

O documento já sinaliza o que é tese e o que é demonstrado. O que importa para o produto:

Blocos	Princípio	Consequência no Loop App
02, 06, 07, 09	Propriedades emergem das relações; combinar análise das partes com a do todo	O modelo de dados guarda **relações** como entidade de primeira classe, não só nós
05	Simplificar ajuda, mas esconde relações; explicitar o que ficou fora	Cada diagrama guarda o **julgamento de fronteira**: o que está dentro, fora e por quê
10, 18	Modelar causa-efeito para achar pontos de alavancagem antes de intervir	A simulação marca **gargalos e alavancas** e liga cada uma a uma intervenção proposta
21	Iceberg, diagramas causais e arquétipos → pontos de intervenção → plano de ação	Fluxo do produto: evento → padrão → estrutura → modelo mental → intervenção → backlog
24	Automatizar com agentes altera a autonomia de quem trabalha	Cada intervenção com agente tem campo de **impacto humano** (autonomia, habilidade) e aprovação explícita
25	Compreensão é parcial; soluções precisam ser testadas	Todo número tem **nível de evidência A/B/C/D**; hipóteses viram itens de validação
26, 27	A ferramenta LOOP ainda não tem método integrado nem validação	O produto deve **coletar evidência de uso** (antes/depois) para validar o método. Fato, não marketing

1.3 PDF técnico (slides University of Hull, 25 páginas)

Conteúdo extraído das imagens (fatos [B]):

- **Conceitos-chave:** arquétipo de sistema, pensamento sistêmico (modelo do iceberg), engenharia de sistemas, Stafford Beer.
- **Relação entre participantes:** unitária, pluralista, coerciva.
- **Conceito de sistema:** subsistemas, ambiente, propriedades sistêmicas, parte de sistema maior.
- **Julgamentos de fronteira** (Ulrich/Midgley).
- **Mudanças na engenharia de sistemas:** aprendizagem experiencial, visão holística da situação, **consultoria de processo** com facilitação e participação, modelagem flexível.
- **Soft OR / PSMs** (Ackoff, Checkland, Eden, Rosenhead, Mingers); lógica: situação complexa → grupo → **mapeamento** (rich picture) → **modelagem** → discussão → planos de ação → muda a situação (ciclo).
- **Modelo de definição e atividades** (Trussell Trust) com medidas de desempenho: **eficácia, eficiência, efetividade** (os "3 Es" da SSM).
- **Arquétipos:** fixes that fail, shifting the burden, escalation, eroding goals, limits to success, success to the successful, growth & underinvestment, accidental adversaries, tragedy of the commons.

- **Cenários de projeto:** soft OR clássico (workshop), coaching estratégico (1:1), pesquisa aplicada, engajamento de stakeholders.
- **VSM:** cinco funções — S1 operações, S2 coordenação, S3 controle, S4 inteligência/ambiente, S5 política/identidade; E (ambiente), O (operação), M (metassistema).
- **Papel do pensamento sistêmico:** Tipo A modela *sistemas* (baseline, design, teórico, processo); Tipo B modela *mudanças* (transformações desejáveis, projetos de mudança).
- **Visões de mundo alternativas** (exemplo do cinema: negócio × comunidade).
- **Diagramas causais de exemplo:** proteção infantil (Munro Review), penas de prisão (Rouwette).
- **Organizações vistas como sistemas** (referente: organização, unidade, serviço, processo, projeto de mudança; espectro descritivo → criativo).
- Leituras: Checkland (1999, 2006), Rosenhead & Mingers (2001), Jackson (2003), Hindle (2011).

Consequência de design: o catálogo de tipos de diagrama do produto é derivado diretamente desse conteúdo (§3.5, entidade `Model.kind`): `rich_picture`, `causal_loop`, `influence`, `archetype`, `vsm`, `activity_model`, `flow_map` (metrô), `iceberg`. Os tipos A/B de Hindle viram o campo `Model.purpose = system | change`.

1.4 DIAGRAMA_V2.html — Mapa de Fluxos CleanSea

Protótipo funcional (HTML + SVG + JS, sem dependências) da operação "da solicitação ao certificado".

- **Estações:** N navio/armador/agente (externo) · A análise comercial · C confirmação do cliente (atraso puro) · P programação e anuências · Y autoridades e porto (externo, controle) · K coleta a bordo (capacidade = nº de equipes) · T transporte · R tratamento e destinação (externo, espera pura) · D documentação e controle (com retrabalho).
- **Três linhas (fluxos):** físico (resíduo/veículo), informação (OS, cotação, registros), controle (anuência, devolução).
- **Simulação:** eventos discretos, 30 dias simulados, chegadas exponenciais, tempos Erlang, filas e uso por estação, alerta quando espera > 2,3× a referência.
- **Parâmetros base (fictícios, editáveis):** `lambda` 3,6 pedidos/dia · `confirm` 0,25 dia · `clear` 0,25 dia · `crews` 2 · `rework` 8%.
- **Cenários:** referência · atraso de informação (4×) · pico (+40%) · falta de equipe (1) · anuência lenta (36 h) · retrabalho (35%).
- **Design system próprio:** gramática do metrô (Beck 1931), 4 cores com papéis fixos, octilinear, indicador de estado por pontos, contraste calculado, tema claro/escuro, `prefers-reduced-motion`.

Avaliação: é a peça mais madura e prova a proposta de valor (o cliente "vê" a operação e testa "e se"). Limitação: estações, parâmetros e cenários estão **embutidos no código**. Para servir a qualquer cliente, o modelo precisa virar **dados** (schema) e o motor de simulação precisa virar um **módulo reutilizável**. É o principal requisito técnico desta análise (FRD §5.5).

Pontos de alavancagem já visíveis no modelo [C]: P (anuência) é gargalo de **controle**, não de capacidade; C é atraso externo que ociosa o campo; D com retrabalho alto passa de 100% de uso. Essas são exatamente as intervenções candidatas para os agentes A1/A2 (triagem e extração/rascunho reduzem retrabalho em D e tempo em A).

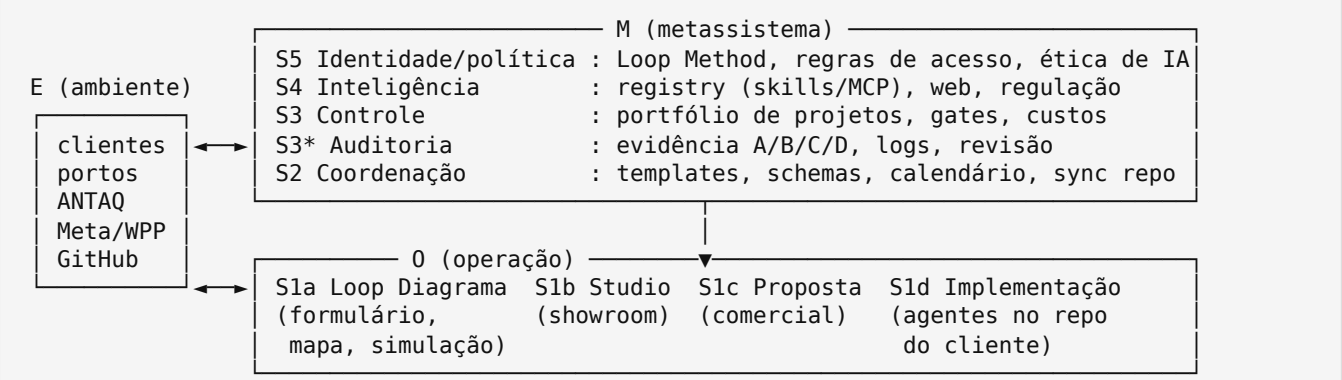
1.5 Pesquisa externa (web)

Tema	Achado	Uso no design
Regulação de resíduos de navios no Brasil [B]	Procedimentos de portos citam a Resolução ANTAQ nº 99/2023 e as Res. 4.828/2016, 4.865/2016 e 2.650/2012; a autoridade portuária responde à ANTAQ pela habilitação dos prestadores; dados dos prestadores vão ao módulo PRFD do GISIS (IMO). Não confirmei o texto da Res. 99/2023.	Estações P/Y/D e o certificado precisam de regras **por porto** versionadas; validar com o cliente e com o texto oficial
Software de VSM [B]	Existem ferramentas (VSMod, Viplan, VSM Toolbox, projeto UVa, "Live Cybernetics"), em geral acadêmicas ou sem manutenção verificada	Diferencial do Loop App: VSM + simulação + backlog ligado ao repositório, não apenas diagnóstico
WhatsApp Business Cloud API [B]	Cobrança por mensagem desde jul/2025; preços por categoria; valores para o Brasil divergem entre fontes; publicação em BRL prevista para 2026	Custo de E_6 = **A DEFINIR** a partir da tabela oficial da Meta
Auth multi-tenant em Cloudflare [B]	Better Auth tem suporte a D1 e plugin multi-tenant; no OpenNext, acesso ao binding via `getCloudflareContext()`	Opção recomendada para Fase 0 (§3.6), mantendo a stack atual do repositório

---

2. Leitura sistêmica da própria plataforma (VSM)

A Loop Consulting aplica o VSM aos clientes. Aplicado à própria plataforma, mostra quais funções o software precisa ter:



Julgamento de fronteira (v0.1): dentro = Loop App, projetos, modelos, simulações, backlog, sync com GitHub. Fora (integra, não controla) = sistemas internos do cliente, autoridades portuárias, Meta, provedores de modelo. Excluído nesta fase = faturamento, contratos jurídicos, ERP do cliente.

Relação entre participantes: CleanSea ↔ Loop tende a **pluralista** (interesses compatíveis, valores diferentes) [C]. O produto deve, por isso, dar espaço para **comentários e divergência registrada** no modelo, não só aprovação.

---

3. System design

3.1 Requisitos

Funcionais (resumo; detalhe no FRD §5)

1. Multi-tenant: um tenant por cliente; áreas privada, compartilhada, studio e proposta.
2. Abrir projeto a partir de template + schemas; criar ou ligar o repositório GitHub do cliente.
3. Formulários de coleta (intake) que alimentam o modelo.
4. Editor/visualizador de modelos sistêmicos (tipos do §1.3) com evidência A/B/C/D.
5. Motor de simulação data-driven (generalização do Mapa de Fluxos) com cenários.
6. Studio (showroom) e proposta comercial derivados do modelo.
7. Comunicação desenvolvedor ↔ cliente (comentários, dúvidas, notificações por e-mail e, depois, WhatsApp).
8. Sincronização com o repositório do cliente (schemas, decisões, backlog).
9. Consumo do registry já existente (`storage/indexes/registry.json`) para escolher skills/MCP/plugins por projeto.

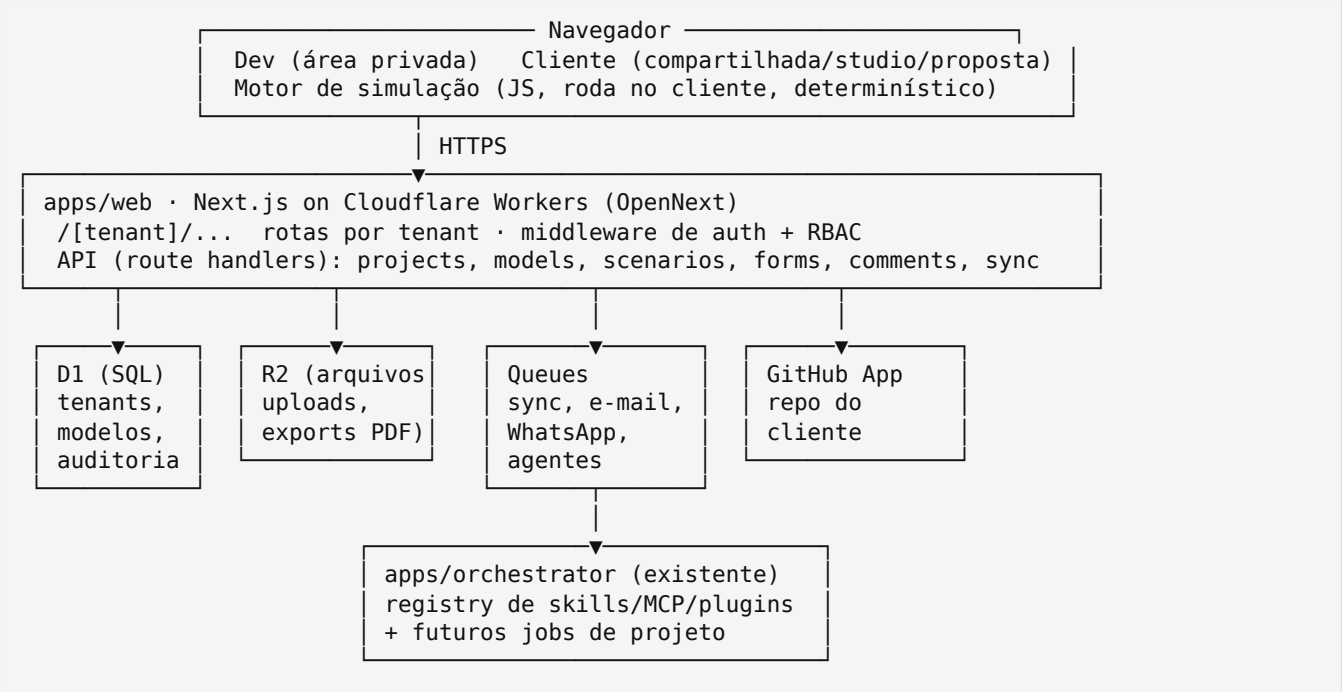
Não funcionais

Atributo	Meta inicial	Fonte
Escala	≤ 20 tenants, ≤ 50 usuários ativos, ≤ 10 projetos ativos no 1º ano	[C] estimativa de consultoria pequena
Latência	página < 2 s p95; simulação de 30 dias < 1 s no navegador	[C]; o protótipo já roda no cliente
Disponibilidade	99,5% mensal (horário comercial crítico)	[C]
Isolamento	dado de um tenant nunca visível a outro (teste automatizado)	obrigatório
Segurança	sem segredos no repositório; MFA para desenvolvedores; logs de acesso	obrigatório
Acessibilidade	WCAG 2.2 AA, tema claro/escuro, movimento reduzido	já praticado no protótipo
Custo	infraestrutura serverless com escala a zero	[C]
Idioma	pt-BR na interface; dados de pedidos podem ter outros idiomas	[D] pergunta do João

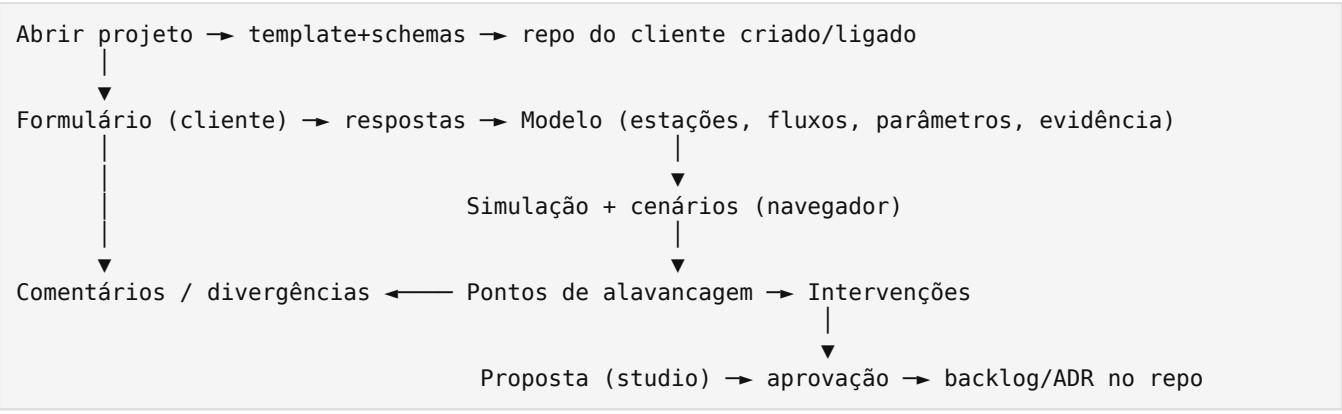
Restrições

- Equipe pequena (1–2 desenvolvedores) [C].
- Stack já presente: monorepo npm, Next.js 16 + React 19 + Tailwind 4, OpenNext em Cloudflare Workers, orquestrador Node sem dependências.
- Deploy no Cloudflare ainda bloqueado por credencial (pendência de ambiente, não de arquitetura).

3.2 Arquitetura de alto nível



Fluxo principal (Loop Diagrama):



3.3 Componentes

Componente	Responsabilidade	Reuso
`apps/web`	UI, rotas por tenant, API	Template já no repo
`packages/model-schema` (novo)	JSON Schema + tipos TS de `Model`, `Station`, `Flow`, `Scenario`, `Evidence`	Fonte única; consumida por web, orquestrador e repositório do cliente
`packages/sim-engine` (novo)	Simulação de eventos discretos extraída do DIAGRAMA_V2, determinística com seed	Código do protótipo
`packages/flow-map` (novo)	Renderizador SVG do mapa de metrô + design tokens	Design system do protótipo
`apps/orchestrator`	Registry (existente); ganha jobs de sync de projeto	Existente
GitHub App "Loop"	Cria/espelha repo do cliente, abre PRs com schemas/backlog	Novo
Workers de fila	e-mail, WhatsApp, sync, chamadas de agentes	Novo

3.4 Sincronização com o repositório do cliente

Decisão: **a plataforma é a fonte da verdade do modelo; o repositório é a fonte da verdade do código.** O sync é unidirecional por tipo de artefato e sempre por **pull request**, nunca push direto.

Artefato	Direção	Formato no repo
Schemas do projeto, modelo, cenários	plataforma → repo (PR)	`loop/model/*.json` validados pelo schema
Decisões (ADR) e intervenções aprovadas	plataforma → repo (PR)	`loop/decisions/NNNN-*.md`
Backlog	plataforma → repo	Issues com label `loop`
Estado de execução (PRs, CI, issues fechadas)	repo → plataforma (webhook)	leitura

Conflito: se o arquivo no repo mudou desde o último sync, a plataforma não sobrescreve; abre PR com diff e marca o projeto como "divergente".

3.5 Modelo de dados (núcleo)

```
Tenant(id, slug, name, status)
User(id, email, name, mfa)                      Membership(user_id, tenant_id, role)
  role ∈ {owner_dev, dev, client_admin, client_member, viewer}
Project(id, tenant_id, name, template_id, repo_url, phase, boundary_json)
Form(id, project_id, schema_json, status)       FormResponse(id, form_id, user_id, data_json,
submitted_at)
Model(id, project_id, kind, purpose, version, data_json, created_by)
  kind ∈ {flow_map, causal_loop, influence, archetype, vsm, rich_picture, activity_model,
iceberg}
  purpose ∈ {system, change}                    -- Tipo A / Tipo B (Hindle)
Parameter(id, model_id, key, value, unit, evidence, source_note)   evidence ∈ {A,B,C,D}
Scenario(id, model_id, name, overrides_json)    SimulationRun(id, scenario_id, seed, horizon,
result_json)
Intervention(id, project_id, target_ref, description, human_impact, status, approved_by)
Comment(id, project_id, target_ref, user_id, body, area, stance)   stance ∈ {note, question,
disagree, agree}
Proposal(id, project_id, version, content_json, status, approved_at)
SyncEvent(id, project_id, direction, artifact, pr_url, status, at)
AuditLog(id, tenant_id, user_id, action, target, at)
```

Regras: toda tabela de negócio carrega `tenant_id`; toda consulta filtra por ele no repositório de dados (não na rota). `Parameter.evidence` é obrigatório. `Model` é versionado (imutável por versão).

3.6 Autenticação, áreas e permissões

- **Login individual** (link mágico por e-mail ou OAuth Google/Microsoft) — substitui a "senha por rota" das notas. A URL `/[tenant]` continua existindo; a senha não.
- MFA obrigatório para `owner_dev` e `dev`.
- Matriz de áreas:

Área	owner_dev	dev	client_admin	client_member	viewer
Gerenciamento (privada)	RW	RW	—	—	—
Compartilhada	RW	RW	RW	RW	R
Studio	RW	RW	R + rodar simulação	R + rodar simulação	R
Proposta	RW	RW	R + aprovar	R	R

- Implementação recomendada: Better Auth com D1 e plugin multi-tenant [B]; alternativa: Cloudflare Access para a área privada.

3.7 API (REST, JSON)

```
POST   /api/tenants/{t}/projects                        criar projeto a partir de template
POST   /api/tenants/{t}/projects/{p}/repo               ligar/criar repo do cliente
GET    /api/tenants/{t}/projects/{p}/models?kind=
POST   /api/tenants/{t}/projects/{p}/models             nova versão (valida schema)
POST   /api/tenants/{t}/models/{m}/scenarios
POST   /api/tenants/{t}/scenarios/{s}/runs              grava resultado vindo do navegador (seed
+ resultado)
GET|POST /api/tenants/{t}/projects/{p}/forms[/{f}/responses]
GET|POST /api/tenants/{t}/projects/{p}/comments
POST   /api/tenants/{t}/proposals/{id}/approve
POST   /api/webhooks/github                              assinatura verificada
```

Erros: corpo `{ code, message, details }`; 403 para área sem permissão; 409 para versão de modelo desatualizada; idempotência por `Idempotency-Key` em POST de runs, aprovações e sync.

3.8 Armazenamento e trade-offs

Decisão	Opção A (recomendada agora)	Opção B	Por quê
Banco	**Cloudflare D1** (SQLite)	Prisma Postgres (E_10)	Mesma plataforma do deploy, custo a zero, volume pequeno. Migrar para Postgres quando houver RAG/pgvector, relatórios pesados ou > 10 GB. Manter SQL portável
Simulação	**No navegador**	Servidor	O protótipo já roda no cliente, interação instantânea, sem custo de servidor. Gravar seed + parâmetros garante reprodução
Modelo	**JSON versionado validado por schema**	Grafo em tabelas	Diagramas variam muito por tipo; JSON + schema é flexível e versionável no Git do cliente
Sync	**PR via GitHub App**	Push direto	Revisão humana, auditoria, sem sobrescrever trabalho do cliente
Tempo real	Polling / revalidação	WebSocket (Durable Objects)	Poucos usuários simultâneos; adiar até haver necessidade medida
Agentes CleanSea	No **repositório da CleanSea**, consumindo o mesmo schema	Dentro do Loop App	Separa produto da Loop do produto do cliente; o Loop App só orquestra e visualiza

3.9 Escala, confiabilidade e observabilidade

- Carga estimada [C]: dezenas de requisições/minuto em pico. Workers escalam horizontalmente; D1 suporta com folga.
- Falhas externas (GitHub, e-mail, WhatsApp, modelo de IA): sempre via fila, retry com backoff exponencial, dead-letter e alerta.
- Backup: export diário do D1 para R2; o modelo também vive no repo do cliente (segunda cópia). Testar restauração por trimestre.
- Observabilidade: logs estruturados por `tenant_id`/`request_id`; métricas: tempo de página, falhas de sync, custo de IA por projeto; alerta em falha de sync > 1 h.
- Teste obrigatório de isolamento entre tenants em CI.

3.10 O que revisitar com o crescimento

1. D1 → Postgres quando entrar RAG/busca semântica ou analytics pesado (E_5).
2. Simulação no servidor se modelos passarem de ~10^5 eventos ou exigirem lote/otimização.
3. Colaboração em tempo real (Durable Objects) se houver workshops ao vivo com vários participantes.
4. Separar o Studio em app próprio se o cliente quiser domínio dedicado (E_1/E_8).

---

4. PRD — Product Requirements Document

4.1 Problema

Propostas de consultoria em IA são abstratas: o cliente não vê a própria operação, não testa alternativas e decide por confiança. A implementação depois fica desconectada da análise. Para a CleanSea, o ciclo pedido → cotação → coleta → certificado tem gargalos (anuência, confirmação, retrabalho de documentos) que ninguém vê de ponta a ponta [C].

4.2 Objetivos

#	Objetivo	Métrica
O1	Cliente visualiza a empresa como sistema e testa cenários antes de contratar	≥ 1 sessão de simulação do cliente antes da aprovação da proposta
O2	Converter proposta em serviço com base no modelo	Proposta aprovada **dentro** da plataforma
O3	Ligar análise à execução	100% das intervenções aprovadas viram issue/ADR no repo do cliente
O4	Trocar suposição por evidência	% de parâmetros com evidência A/B sobe a cada ciclo (meta inicial: ≥ 60% antes da Fase 3)
O5	Padronizar o método entre clientes	2º cliente aberto a partir de template sem código novo

4.3 Fora do escopo (v1)

- Os agentes A0–A3, o app Expo e o MCP **da CleanSea** são produtos do cliente: ficam no repositório dela. O Loop App planeja, visualiza e acompanha (E_8 é a ponte).
- Faturamento, contrato, ERP.
- Otimização automática de cenários.

4.4 Personas

- **Desenvolvedor/consultor Loop** — estrutura o projeto, modela, propõe, implementa. Precisa de área privada, templates e sync.
- **Gestor do cliente (ex.: João, CleanSea)** — fornece informação, valida o modelo, aprova proposta. Pouco tempo; usa celular.
- **Operador do cliente** — responde formulários específicos, comenta etapas que conhece.

4.5 Histórias de usuário (prioridade MoSCoW)

ID	Como...	quero...	para...	Prioridade
US1	dev	abrir um projeto a partir de template e ligar ao repo do cliente	começar sem setup manual	Must
US2	cliente	entrar com meu e-mail e ver só o meu projeto	ter segurança	Must
US3	cliente	responder ao formulário de intake no celular	fornecer dados sem reunião	Must
US4	dev	transformar respostas em um mapa de fluxos com parâmetros e evidência	ter um modelo comum	Must
US5	cliente	rodar cenários ("e se a anuência demorar 6×?")	entender gargalos	Must
US6	cliente	comentar, perguntar e discordar em qualquer estação	registrar minha visão	Must
US7	dev	marcar pontos de alavancagem e propor intervenções com impacto humano	justificar a solução	Should
US8	cliente	ver a proposta ligada ao modelo e aprová-la	decidir com base no que vi	Should
US9	dev	publicar intervenções aprovadas como PR/issues no repo do cliente	executar sem retrabalho	Should
US10	dev	escolher skills/MCP do registry para o projeto	montar a stack de agentes	Could
US11	cliente	receber avisos por e-mail (depois WhatsApp)	acompanhar sem entrar todo dia	Could
US12	dev	criar diagramas causais, VSM e arquétipos além do mapa de fluxos	aplicar o método completo	Could (Fase 2)

4.6 Fases e marcos

Fase	Conteúdo	Critério de saída
0 · Fundação	auth, tenants, RBAC, template de projeto, GitHub App, `model-schema`	Teste de isolamento verde; tenant CleanSea criado; repo ligado
1 · Loop Diagrama	formulário de intake, `sim-engine` + `flow-map` extraídos do DIAGRAMA_V2, cenários, comentários	Mapa CleanSea (1 porto) renderizado a partir de dados; cliente rodou cenário
2 · Studio e proposta	showroom, proposta versionada, aprovação, sync de intervenções, tipos extras de diagrama	Proposta aprovada na plataforma; issues criadas no repo
3 · Operação de agentes	acompanhamento de E_2–E_7 no repo CleanSea, e-mail/WhatsApp, painel de evidência	Ciclo pedido→cotação medido antes/depois

Datas: **A DEFINIR** (dependem de capacidade e das respostas do cliente).

4.7 Riscos

Risco	Prob.	Impacto	Mitigação
Dados do cliente não chegam ou chegam incompletos	Alta	Alto	Formulário curto por etapa; evidência C/D explícita; simulação funciona com fictícios
Cliente confunde simulação fictícia com previsão	Média	Alto	Rótulo permanente "valores fictícios" e nível de evidência por parâmetro
Vazamento entre tenants	Baixa	Crítico	`tenant_id` no acesso a dados + teste em CI + auditoria

Risco	Prob.	Impacto	Mitigação
Escopo do Loop App absorver o produto CleanSea	Alta	Médio	Separação de repositórios (§4.3)
Regras portuárias mudam ou variam por porto	Alta	Médio	Regras versionadas por porto; começar com 1 porto
Custo de WhatsApp/IA imprevisto	Média	Médio	Fila com orçamento por projeto; tabela oficial da Meta antes de E_6
Automação reduz autonomia da equipe do cliente (bloco 24)	Média	Médio	Campo de impacto humano + aprovação explícita

4.8 Perguntas abertas (entrada do cliente — lista do João)

Organograma e envolvidos · como chega um pedido hoje · volume por dia/mês · tempo da mensagem à cotação · onde estão as regras de cada porto e quem as atualiza · existe tabela porto × serviço · quem aprova mudanças · nos 28 portos a operação é própria ou de parceiros · e-mail Google ou Microsoft · WhatsApp já é Business · idioma dos pedidos · retenção de mensagens · o que já deu errado e custou caro (multa, atraso, cliente perdido) · suporte pós-entrega · KPIs. Entregáveis do cliente: tabela de preços, documentos e regras, mensagens de exemplo, organograma, acesso ao e-mail operacional.

Cada resposta deve atualizar um `Parameter` (evidência A) ou uma regra de porto.

---

5. FRD — Functional Requirements Document

Notação: EARS + RFC 2119. Cada requisito tem ID, prioridade e critério de aceite.

5.1 Tenancy e autenticação (Fase 0)

- **FR-AUTH-01 (Must)** O sistema MUST autenticar cada pessoa individualmente (link mágico ou OAuth).
- **FR-AUTH-02 (Must)** IF o papel é `owner_dev` ou `dev`, THEN o sistema MUST exigir MFA.
- **FR-AUTH-03 (Must)** WHEN um usuário acessa `/[tenant]/…`, THEN o sistema MUST verificar `Membership` nesse tenant e MUST responder 404 se não houver.
- **FR-AUTH-04 (Must)** Toda leitura e escrita de dados de negócio MUST filtrar por `tenant_id` no acesso a dados.
- **FR-AUTH-05 (Must)** O sistema MUST registrar em `AuditLog` login, convite, mudança de papel, aprovação e sync.
- **FR-AUTH-06 (Should)** `client_admin` SHOULD poder convidar `client_member` e `viewer` do próprio tenant.

```
Scenario: Isolamento entre tenants
  GIVEN o usuário U é membro somente do tenant "cleansea"
  WHEN U requisita /api/tenants/outro/projects
  THEN a resposta é 404
  AND nenhum dado de "outro" aparece em logs de resposta
```

5.2 Áreas e visibilidade

- **FR-AREA-01 (Must)** O sistema MUST aplicar a matriz do §3.6.
- **FR-AREA-02 (Must)** Conteúdo da área privada MUST NOT aparecer para papéis de cliente, nem por busca, nem por API.

- **FR-AREA-03 (Should)** WHEN um dev publica um item da área privada na compartilhada, THEN o sistema SHOULD criar uma cópia versionada e registrar quem publicou.

5.3 Projeto, template e repositório

- **FR-PROJ-01 (Must)** WHEN um dev cria um projeto, THEN o sistema MUST instanciar o template escolhido (schemas, formulários, modelo inicial vazio).
- **FR-PROJ-02 (Must)** O sistema MUST permitir ligar um repo existente ou criar um novo via GitHub App com permissão mínima.
- **FR-PROJ-03 (Must)** O sync plataforma → repo MUST ocorrer somente por pull request.
- **FR-PROJ-04 (Must)** IF o arquivo de destino mudou no repo desde o último sync, THEN o sistema MUST NOT sobrescrever e MUST marcar o projeto como "divergente".
- **FR-PROJ-05 (Should)** WHEN o repo recebe push, PR ou fechamento de issue com label `loop`, THEN o sistema SHOULD atualizar o estado da intervenção ligada.
- **FR-PROJ-06 (Must)** Webhooks MUST ter assinatura verificada.

5.4 Formulários (intake)

- **FR-FORM-01 (Must)** Formulários MUST ser definidos por JSON Schema do template.
- **FR-FORM-02 (Must)** O formulário MUST funcionar em tela de 375 px e MUST preservar respostas após falha de envio.
- **FR-FORM-03 (Must)** WHEN uma resposta mapeia para um parâmetro, THEN o sistema MUST atualizar o `Parameter` com evidência A e a origem.
- **FR-FORM-04 (Should)** O sistema SHOULD salvar rascunho automaticamente.

5.5 Modelos e simulação (Loop Diagrama)

- **FR-MOD-01 (Must)** O modelo `flow_map` MUST ser descrito em dados: estações (papel, externo/interno, tipo de espera: fila | atraso), fluxos (físico, informação, controle), trechos, parâmetros e cenários. Nada embutido no código.
- **FR-MOD-02 (Must)** O sistema MUST renderizar o mapa CleanSea atual a partir desses dados com resultado visual equivalente ao DIAGRAMA_V2.
- **FR-MOD-03 (Must)** Todo `Parameter` MUST ter `evidence` ∈ {A, B, C, D} e a interface MUST mostrar esse nível.
- **FR-MOD-04 (Must)** IF algum parâmetro tem evidência C ou D, THEN a simulação MUST mostrar o aviso "valores não validados".
- **FR-MOD-05 (Must)** A simulação MUST ser determinística para o mesmo seed, parâmetros e horizonte.
- **FR-MOD-06 (Must)** O sistema MUST calcular, por estação, espera média, uso e fila, e MUST sinalizar gargalo quando a espera passar de um limite configurável (padrão 2,3× a referência).
- **FR-MOD-07 (Must)** Usuários de cliente MUST poder rodar e comparar cenários sem alterar o modelo base.
- **FR-MOD-08 (Should)** Modelos SHOULD ser versionados; uma nova versão MUST NOT apagar resultados de simulação anteriores.
- **FR-MOD-09 (Should)** Cada modelo SHOULD guardar o julgamento de fronteira (dentro, fora, excluído, motivo).
- **FR-MOD-10 (Could)** O sistema COULD oferecer os tipos `causal_loop`, `archetype`, `vsm`, `rich_picture`, `activity_model`, `iceberg`, `influence` (Fase 2), reaproveitando o design do diagrama CLD já no repositório (`docs/diagrama-cld`).
- **FR-MOD-11 (Must)** A interface MUST respeitar `prefers-reduced-motion`, ter tema claro/escuro e nunca usar só cor para transmitir informação.

```
Scenario: Cenário de anuência lenta mostra gargalo de controle
  GIVEN o modelo CleanSea base com 2 equipes
  WHEN o cliente aplica o cenário "Anuência lenta" (clear = 1,5 dia)
  AND roda 30 dias simulados
  THEN a estação "Programação e anuências" recebe alerta de gargalo
  AND a estação "Coleta a bordo" não recebe alerta

Scenario: Reprodutibilidade
  GIVEN um cenário com seed 11
  WHEN a simulação roda duas vezes
  THEN os dois resultados são idênticos
```

5.6 Comentários e comunicação

- **FR-COM-01 (Must)** Usuários MUST poder comentar em projeto, modelo, estação, parâmetro ou intervenção, com `stance` (nota, pergunta, discordância, concordância).
- **FR-COM-02 (Must)** Comentários da área privada MUST NOT ser visíveis a clientes.
- **FR-COM-03 (Should)** WHEN um comentário é uma pergunta ao cliente, THEN o sistema SHOULD notificar por e-mail.
- **FR-COM-04 (Could)** O sistema COULD notificar por WhatsApp Business (Fase 3), com opt-in registrado e orçamento por projeto.

5.7 Intervenções, studio e proposta

- **FR-INT-01 (Should)** Um dev SHOULD poder criar uma intervenção ligada a um gargalo ou parâmetro, com descrição, impacto humano (autonomia, habilidade) e componentes do registry escolhidos.
- **FR-INT-02 (Must)** IF uma intervenção usa agente que acessa arquivos, rede, credenciais ou sistemas do cliente, THEN ela MUST exigir aprovação explícita do `client_admin`.
- **FR-PROP-01 (Should)** A proposta SHOULD ser gerada a partir do modelo, dos cenários e das intervenções, e responder: o que ganho e perco, quanto custa, riscos, problemas resolvidos, impacto em vendas/margem.
- **FR-PROP-02 (Should)** WHEN o `client_admin` aprova a proposta, THEN o sistema SHOULD congelar a versão e enviar as intervenções ao repo (FR-PROJ-03).
- **FR-STU-01 (Should)** O studio SHOULD apresentar a solução em modo leitura com simulação interativa, adequado a celular.

5.8 Registry e stack de agentes

- **FR-REG-01 (Could)** O sistema COULD listar componentes de `storage/indexes/registry.json` com filtros e risco de segurança, para seleção por projeto.
- **FR-REG-02 (Must)** O sistema MUST NOT executar componentes do registry; apenas registrar a escolha e gerar instrução de instalação para revisão.

5.9 Requisitos não funcionais verificáveis

ID	Requisito	Verificação
NFR-01	Página p95 < 2 s	Lighthouse / RUM
NFR-02	Simulação 30 dias < 1 s em celular médio	Teste de desempenho do `sim-engine`
NFR-03	WCAG 2.2 AA	axe + teclado + leitor de tela
NFR-04	Zero segredos no repositório	secret scan no CI

ID	Requisito	Verificação
NFR-05	Backup diário e restauração testada	runbook + registro trimestral
NFR-06	Isolamento de tenant	teste automatizado em CI

5.10 Rastreabilidade

Requisito	Fonte	Entrega CleanSea
FR-AUTH-, *FR-AREA-*	Notas Loop (áreas, login)	E_8
FR-PROJ-*	Notas Loop (espelhamento com repo)	E_9, E_3
FR-FORM-*	Notas Loop (formulários)	E_7
FR-MOD-*	DIAGRAMA_V2, PDF (tipos de modelo), Baptista 05/18/25	E_5 (base analítica)
FR-COM-*	Notas Loop (dúvidas), PDF (participação, pluralismo)	E_6
FR-INT-02	Baptista 24	E_2
FR-PROP-*	Notas Loop (perguntas do cliente)	—
FR-REG-*	Registry existente	E_2, E_3
(Fase 3, banco)	E_10 Prisma	E_10

---

6. Próximos passos recomendados

1. Validar este documento com o cliente e decidir as datas (§4.6).
2. Enviar a lista do João (§4.8) como **primeiro formulário** do tenant CleanSea.
3. Fase 0: criar `packages/model-schema` e extrair `sim-engine` + `flow-map` do DIAGRAMA_V2, com teste de reprodutibilidade (FR-MOD-05) antes de qualquer UI nova.
4. Resolver a credencial do Cloudflare para habilitar deploy (pendência de ambiente).
5. Confirmar no texto oficial a Resolução ANTAQ nº 99/2023 e as regras do porto-piloto.

---

7. Fontes externas consultadas

- Resolução ANTAQ 2190/2011 (Portos Rio)
- ANTAQ — Manual de instalações portuárias para recepção de resíduos
- ANTAQ — GISIS e a responsabilidade dos portos
- Portonave — Procedimento para retirada de resíduos de embarcações
- Porto Sudeste — Manual de resíduos de embarcação
- IMO — Planos de gestão de resíduos portuários no Brasil e na Jamaica
- Viplan e o Viable System Model (Emerald)
- Metaphorum — diagnóstico rápido com cibernética organizacional (VSMod)
- INSISOC/UVa — plataforma de VSM
- Live Cybernetics (HN Showcase)
- Meta — Preços da WhatsApp Business Platform
- Zenvia — FAQ das novas regras de cobrança do WhatsApp Business
- Calculadora de preços do WhatsApp para o Brasil (Hyperleap)
- better-auth-cloudflare (D1, OpenNext)
- D1 bindings com Next.js em Workers